

CS2106 Operating Systems 2024_25 Semester 2 MIDTERM ASSESSMENT - Confidential

Exam ID: 46838

Exam UID: 6272217410476388161514XX

Post: -

Course Name: -

of Questions: 6

Date and Time of Exam Creation: Tue, Mar 04, 2025 @ 07:29:05

Notices:

1. On-campus Exam: Duration = 1 HR

NATIONAL UNIVERSITY OF SINGAPORE

CS2106 - Operating Systems

Semester /2 : AY 2024/25

Time Allowed: 60 MINUTES

INSTRUCTIONS TO STUDENTS

1. This assessment contains **10** questions, organized into 6 large questions.
2. Please answer **ALL** questions.
3. This is a **SECURE** assessment.
4. A single A4 cheat sheet is permitted.
5. The laptop you are using to answer these questions, ormal, scientific, graphing and financial calculators are allowed. No other devices are allowed.
6. You are not permitted to leave the venue.
7. Please place your student card or identification document (NRIC, driving license, etc.) on the top right hand corner of your desk.

8. Please switch off your personal devices with communication features and leave it on the floor next to your desk at all times.
9. If you wish to communicate with an invigilator, go to the washroom, or leave before the end of the assessment, please raise your hand to inform the invigilator.
10. You will be liable for disciplinary action which may result in expulsion from the University if you are found to have contravened any of the clauses below,
- Violation of the NUS Code of Student Conduct (in particular the part on Academic, Professional and Personal Integrity), NUS IT Acceptable Use Policy or NUS Examination rules.
 - Possession of unauthorized materials/electronic devices.
 - Bringing your mobile phone or any storage/communication device with you to the washroom.
 - Unauthorized communication e.g. with another student.
 - Reproduction of any exam materials outside of the exam venue.
 - Photography or videography within the exam venue.
 - Plagiarism, giving or receiving unauthorised assistance in academic work, or other forms of academic dishonesty.
11. Once you have completed the assessment,
- Click on the "Exam Controls" button and choose "Submit Exam".
 - Check off "I am ready to exit my exam" and click on "Exit" to upload your answers to the server.
 - You will see a green confirmation window on your screen when the upload is successful.
- Please keep this window on your screen.**
- If you do not see a green window, please disconnect and reconnect your WIFI and try again.
 - Please be reminded that it is your responsibility to ensure that you have uploaded your answers to the Software.
 - Please remain seated quietly in the meantime.
12. By clicking on the "Continue" button below, I acknowledge that I have read and fully understood this Exam Notice.

Thank You

I acknowledge that I have read this notice ____

Exam Creator: dcstanc@nus.edu.sg

Exam Folder: School of Computing\Computer Science\AY 2024-2025 SEM2\CS2106

Current Date and Time: Wed, Sep 24, 2025 @ 21:21:53

Categories Used in Exam:

Categories are not used in this exam

Security Options:

- Secure Exam
- Time Limit: 60 minutes

Enabled Options:

- Calculator
- Numbering
- Notes
- Question Feedback
- Show 5 Min Alarm

Exam Psychometrics:

KR20	Standard Deviation	Mean	Median	Minimum Score	Maximum Score
-0.77	8.08	28.14	29.33	1.33	44.00

Question #: 1

Choose ALL of the statements that are TRUE:

- ✓A. A microkernel OS tends to be more reliable than a monolithic OS, at the expense of performance.
- B. A user program must go through the OS to access **all hardware resources** on the computer.
- C. In a monolithic OS, all components of the OS kernel are compiled together as one large single program.
- ✓D. An OS depends on ISRs to move processes from a blocked state to the ready state
- E. A virtual machine hypervisor is required to virtualize the underlying hardware across processes.

Item ID: 301463 / 2

Item Description: Q1

Item Weight: 2.0

Item Creator: dcstanc@nus.edu.sg

Question #: 2

Choose, from the following list, what would NOT be in a process's Process Control Block (PCB):

- A. Address of next instruction in process to be loaded.
- ✓B. Program code for the current process.

- C. CPU usage information for scheduling decisions.
- ✓D. Contents of I/O registers (registers in an I/O device outside of the CPU to configure and access the device)
- ✓E. Process variables.
- F. Pointer to the current top of stack.

Item ID: 301465 / 2

Item Description: Q2

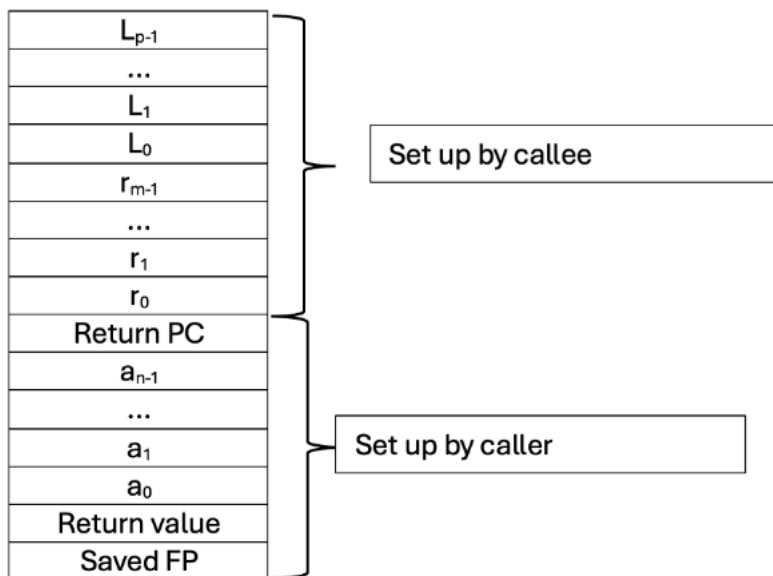
Item Weight: 2.0

Item Creator: dcstanc@nus.edu.sg

Question #: 3

Consider a machine with byte-addressable memory. All addresses, arguments, registers and variables are 4 bytes long.

The diagram below shows the layout of the stack frame set up by the caller and callee, where $a_0 \dots a_{n-1}$ are the n arguments, $r_0 \dots r_{m-1}$ are the m registers saved by the callee, and $L_0 \dots L_{p-1}$ are the p local variables created by the callee.



The algorithm to set up the stack frame above is shown below. At the start, SP is assumed to be pointing to the location in the stack where the next item is to be inserted. As per Tutorial 1, "pushing" to the stack involves reserving space on the stack by saving SP to FP, incrementing SP to reserve space, then using lw/sw to access stack elements relative to FP. For example (shown in "MIPS-like" assembly):

```

addi $fp, $sp, 0      # Copy SP to FP
addi $sp, $sp, 8      # Reserve 2 integers
sw $t0, 0($fp)        # Save first integer to stack
sw $t1, 4($fp)        # Save second integer to stack

```

The algorithm itself is shown here. Note that the callee does not modify FP:

Caller	Callee
1: Save FP on stack	
2: Copy SP to FP	
3: $SP = SP + val_1$ (Reserve space on stack for return value, $a_0, \dots$, Return PC)	
4: Insert arguments $a_0, \dots, a_{n-1}$ to stack	
5: call function	
	6: $SP = SP + val_2$ (Reserve space on stack for m registers and p local variables)
	7: Save m registers $r_0, \dots, r_{m-1}$ to the stack
	8: Assume p local variables $L_0, \dots, L_{p-1}$ on the stack, initialize if necessary.
	... Perform function body ...
	9: $SP = SP - val_2$ (Deallocate stack)
	10: ret (Return to caller)
11: Copy return value from function	
12: Copy FP to SP	
13: Restore FP from stack	

The “call” instruction in line 5 puts the return PC onto the stack location that SP is pointing to before jumping to the callee's starting address. The "call" instruction **does not increment SP**. The “ret” instruction in line 10 copies the contents of the location that SP is pointing to, to PC, thus jumping to the return address. **The “ret” instruction does not decrement SP**.

We are given the following functions:

```

int f(int x, int y) {
    int a, b;
    a = x + y;
    b = b + a;
    return b; // This point
}

```

```

int g(int x, int y) {
    int a = 0;
    a = x + y;
    return a;
}

```

Arguments in functions are inserted into the stack frame in the order that they appear (i.e. argument a_0 is x , argument a_1 is y), and likewise local variables are inserted into the stack frame in the order that they appear. So in function f , local variable a is l_0 , b is l_1 .

After compiling it was found that **function f needs to save 3 registers** onto the stack, while **function g needs to save 4 registers** onto the stack.

Our main is shown below:

```
int main() {  
    g(1, 2); // Line 1  
    f(3, 4); // Line 2  
}
```

Answer questions 3 to 5 based on the information given above.

Question 3

What are the values of val_1 and val_2 when calling function f at Line 2 of our main?

$val_1 = \underline{\quad 1 \quad}$

$val_2 = \underline{\quad 2 \quad}$

Question 4.

We have the following instructions to copy the arguments for function g in line 1 of main:

```
addi $t0, $zero, 1    // Set $t0 to 1  
addi $t1, $zero, 2    // Set $t0 to 2  
sw $t0, disp1($fp)    // Copy $t0 to address given by $fp + disp1  
sw $t1, disp2($fp)    // Copy $t0 to address given by $fp + disp2
```

What are the values of $disp1$ and $disp2$?

$disp1 = \underline{\quad 3 \quad}$ i

$disp2 = \underline{\quad 4 \quad}$

Question 5.

We now execute $main()$ shown above. Suppose that the stack contents is initially all 0. What is the value of local variable b in function f at the line marked “// This point”, as we execute main shown above?

$b = \underline{\quad 5 \quad}$

2. 24
3. 8
4. 12
5. 10

Item ID: 301471 / 5

Item Description: Q3to5

Item Weight: 15.0

Item Creator: dcstanc@nus.edu.sg

Question #: 4

We consider three processes with the following characteristics:

Process	Characteristic	Arrival Time	Priority
A	3CPU, 2IO, 1CPU	0	Low
B	2CPU, 1IO, 2CPU	1	High
C	2CPU, 2IO, 1CPU	2	Medium

(3CPU means that the process uses CPU for 3 ITI, 2IO means it uses IO for 2 ITI, etc)

These processes run on a pre-emptive priority based scheduler. There is only 1 CPU and 1 IO on the system. There is never pre-emption on IO, only on CPU. However if more than one process is about to use IO at the same time, IO is allocated in order of priority.

Answer the questions 6 and 7 based on the information above.

Question 6.

Calculate the response time of the three processes:

Response time of process A = 1 cycles.

Response time of process B = 2 cycles.

Response time of process C = 3 cycles.

Question 7.

Find the turn-around-time (TAT) for the three processes.

TAT of process A = 4 cycles

TAT of process B = 5 cycles

TAT of process C = 6 cycles

1. 0
2. 0
3. 1

4. 12
5. 5
6. 8

Item ID: 301502 / 2

Item Description: Q6to7

Item Weight: 12.0

Item Creator: dcstanc@nus.edu.sg

Question #: 5

Suppose now we have the exact same set of processes running on a **round-robin** system, this time with the higher priority processes getting a longer time quantum, as shown below:

Process	Characteristic	Arrival Time	Time Quantum
A	3CPU, 2IO, 1CPU	0	1
B	2CPU, 1IO, 2CPU	1	3
C	2CPU, 2IO, 1CPU	2	2

As before there is one CPU and one I/O. If more than one process is about to use I/O at the same time, the processes will use I/O in alphabetical order.

Insertions into the **ready queue** follows these rules:

- i. If two processes are ready **at the same time**, they are inserted in alphabetical order. E.g. if A and B are ready at the same time, A is inserted ahead of B and will be picked for running ahead of B.
- ii. New processes are added to the back of the ready queue, EXCEPT:
 - When an existing process A becomes ready and **at the same time** a new process C is introduced, process C is inserted ahead of A. I.e. C will be picked to run ahead of A.
- iii. Otherwise insertion follows standard round-robin policy; a process whose time quantum has expired is inserted at the back of the queue.

Answer questions 8 and 9 based on the information given above:

Question 8.

Response Time of Process A: 1 cycles

Response Time of Process B: 2 cycles

Response Time of Process C: 3 cycles

Question 9.

Find the turn-around-time (TAT) of the three processes:

TAT of Process A: 4 cycles

TAT of Process B: 5 cycles

TAT of Process C: 6 cycles

1. 0
2. 0
3. 2
4. 11
5. 8
6. 8

Item ID: 301504 / 2

Item Description: Q8to9

Item Weight: 12.0

Item Creator: dcstanc@nus.edu.sg

Question #: 6

Consider now the following two processes operating on a shared variable x on a load-store architecture. x is initially 3.

Process A	Process B
$x = x - 2;$	$x = x * 3;$

List down all the possible values produced **without repetition** and in **ascending order** after executing both processes A and B to completion.

For example, if the possible values produced are **9, 3, 4, 3, 7, 9, 5**, list down **3, 4, 5, 7, 9**.

Answer: 1

1. 1, 3, 9|1,3,9|1, 3,9 } 1,3, 9|

Item ID: 301507 / 3

Item Description: Q10

Item Weight: 2.0

Item Creator: dcstanc@nus.edu.sg